

INFORME TÉCNICO DE SISTEMA DE SOFTWARE

SISTEMA DE CONTROL DE INVENTARIO Y PRÉSTAMOS ACADÉMICOS (INVENTARIO-UAM)

Curso:

Programacion Orientada a Objetos II

Carrera:

Ingeniería de Sistemas

Institución:

Universidad Americana (UAM)

Autor:

Jorge Rafael Cubillo Navarro

Fecha:

7 de junio de 2026

RESUMEN EJECUTIVO

El presente informe técnico detalla el diseño, la arquitectura de software y las especificaciones funcionales de *InventAr-UAM*, una aplicación móvil nativa desarrollada para la plataforma Android utilizando el ecosistema moderno de desarrollo de Google: **Kotlin, Jetpack Compose, Room Database y la arquitectura MVVM (Model-View-ViewModel)**.

En este sentido, el sistema ha sido concebido para resolver las problemáticas tradicionales de pérdida de trazabilidad, falta de métricas y vulnerabilidad en la asignación de activos tecnológicos y didácticos dentro de laboratorios y facultades académicas. A través de una interfaz de usuario fluida y reactiva, el sistema permite la administración del ciclo de vida completo de los equipos, el control automatizado de su disponibilidad, la gestión estricta de préstamos e historiales, y la visualización analítica de métricas críticas mediante un tablero de control (*Dashboard*) con soporte para la exportación de registros a formatos portables (CSV).

1. ARQUITECTURA DEL SISTEMA Y TECNOLOGÍAS CORE

Bajo esta premisa, la aplicación está cimentada sobre los principios de la **Clean Architecture** combinada con el patrón de diseño arquitectónico **MVVM**, lo que garantiza una separación clara de responsabilidades, alta mantenibilidad y facilidad para realizar pruebas unitarias o de integración.

Consecuentemente, el flujo de datos y la orquestación de responsabilidades dentro del sistema se estructuran mediante la siguiente jerarquía de capas:

1. Capa de Interfaz de Usuario (UI): Implementada mediante *Jetpack Compose*, gestionando Screens, componentes reutilizables y cuadros de diálogo interactivos.

2. Capa de Lógica de Negocio: Centralizada en los *ViewModels* de inventario, encargados de la gestión reactiva de los estados de la interfaz.

3. Capa de Persistencia y Datos: Basada en el patrón **Repository**, el cual actúa como mediador entre la lógica superior y la base de datos local mediante **Room DB e interfaces DAO**.

- **Model (Capa de Datos):** Representada por las entidades de Room (*Entities.kt*) y las operaciones de persistencia encapsuladas en *InventarioDao*. El repositorio actúa como una fuente única de la verdad (*Single Source of Truth*), abstrayendo el origen de los datos de las capas superiores.
- **ViewModel: *InventarioViewModel*** actúa como el cerebro operativo de la aplicación. Mantiene el estado de la UI expuesto a través de flujos mutables y transforma los datos crudos provenientes de la base de datos en información lista para ser consumida por las vistas. Maneja las corrutinas de Kotlin para asegurar que ninguna operación síncrona en la base de datos bloquee el hilo principal de la interfaz de usuario.

- **View (Capa de UI):** Desarrollada enteramente de forma declarativa con Jetpack Compose.

No retiene lógica de negocio; reacciona estrictamente a los cambios de estado provistos por el ViewModel y emite eventos de usuario hacia el mismo.

2. MODELADO Y PERSISTENCIA DE DATOS (ROOM DATABASE)

La persistencia de datos se resuelve localmente en el dispositivo utilizando un motor SQLite gestionado por la biblioteca de abstracción **Room**. Las relaciones y restricciones se controlan a nivel lógico y de base de datos para asegurar la integridad referencial. El esquema de datos está compuesto por tres estructuras fundamentales:

Tabla: equipo	Tipo de Dato	Restricciones / Descripción
id	Int	Primary Key, Auto-Increment
nombre	String	Not Null. Nombre descriptivo del equipo.
numeroSerie	String	Unique, Not Null. Identificador de hardware.
categoria	String	Not Null. Filtro de clasificación (ej. Laptops, Cables, Kits).
disponible	Boolean	Default true. Bandera de control de estado físico.

Tabla: prestamo	Tipo de Dato	Restricciones / Descripción
id	Int	Primary Key, Auto-Increment
equipoId	Int	Foreign Key (equipo.id) con comportamiento CASCADE.
solicitante	String	Not Null. Nombre/Carnet del estudiante o docente.
fechaPrestamo	String	Not Null. Fecha de salida (ISO u horaria local).
fechaDevolucion	String?	Nullable. Almacena la fecha real en que se entregó el activo.

Tabla: admin	Tipo de Dato	Restricciones / Descripción
id	Int	Primary Key, Auto-Increment
username	String	Unique, Not Null.
passwordHash	String	Not Null. Credenciales locales de acceso técnico.

3. ESPECIFICACIÓN DE MÓDULOS FUNCIONALES IMPLEMENTADOS

Módulo 1: Gestión de Equipos (CRUD Completo)

Este módulo centraliza el control físico de los activos y ofrece interfaces dinámicas para evitar la manipulación errónea de datos:

- **Registro de Equipos:** A través de un cuadro de diálogo flotante (`AddEquipoDialog`), el administrador ingresa los metadatos requeridos. El sistema valida en tiempo real que los campos obligatorios no estén vacíos y que el número de serie mantenga un formato cohesivo antes de realizar la inserción segura en Room.
- **Edición de Equipos:** Permite seleccionar un equipo existente y desplegar sus valores actuales en un formulario de modificación. Al confirmar, invoca el método `updateEquipo` en el repositorio, actualizando los campos sin alterar el historial previo de préstamos asociados a su ID único.
- **Eliminación de Registros:** Incorporado mediante una acción explícita en la tarjeta contenedora del equipo. Incluye una validación de seguridad: si un equipo está actualmente bajo la condición de "Prestado", el sistema restringe su borrado para mitigar inconsistencias de base de datos.
- **Consulta General Dinámica:** Una vista en lista que renderiza tarjetas personalizadas para cada equipo, mostrando visualmente un distintivo de disponibilidad (Verde para Disponible / Rojo para Prestado).

Módulo 2: Gestión de Préstamos y Control de Trazabilidad

Diseñado bajo un flujo estrictamente transaccional para garantizar que un activo no sufra de asignaciones duplicadas:

- **Registro de Préstamos:** El diálogo `RegistrarPrestamoDialog` filtra automáticamente los equipos e interactúa únicamente con aquellos que posean la bandera `disponible = true`. Al procesar el préstamo, se genera el registro en la tabla `prestamo` con la fecha y hora actual, y de forma atómica se actualiza el estado del equipo a `false`.
- **Control de Disponibilidad:** La aplicación restringe e imposibilita la creación de un flujo de salida para cualquier activo que ya se encuentre ocupado, bloqueando botones de interacción en la UI.
- **Registro de Devoluciones:** El historial de movimientos activos expone un botón de acción "Devolver" para los elementos pendientes. Su activación sella el registro del préstamo insertando la fecha de retorno actual en el campo `fechaDevolucion` y revierte la bandera del equipo a `disponible = true`.
- **Historial de Movimientos:** Un listado cronológico inverso que detalla el flujo histórico de quién utilizó cada equipo, cuándo salió y si ya ha sido devuelto satisfactoriamente al almacén.

Utilizando el sistema de componentes compartidos de Android (*Android Intents*), este texto se envía a través de un proveedor de archivos nativo, permitiendo al administrador guardar el archivo `.csv` en el almacenamiento local del dispositivo, enviarlo por correo electrónico institucional o subirlo a la nube.

C. Generación de Estadísticas Gráficas Reactivas

En lugar de depender de librerías externas que aumenten el peso del paquete de distribución (.apk), el tablero analítico dibuja gráficos de barras horizontales personalizados utilizando elementos nativos de Jetpack Compose (Box con modificadores de ancho relativo). El sistema evalúa proporcionalmente la cantidad de equipos por cada categoría y dibuja una barra cuya longitud visual corresponde exactamente al porcentaje representativo en el inventario total.

D. Autenticación Local de Administradores

El acceso a las funcionalidades de escritura, edición y control de inventario está protegido por una pantalla de inicio de sesión segura (LoginScreen). Al iniciar la aplicación, el motor de navegación evalúa una bandera booleana global de sesión (isLoggedIn). Si el estado es falso, el NavHost bloquea las pantallas operativas y redirige el flujo al inicio de sesión. La verificación compara las contraseñas ingresadas con el hash seguro resguardado en la entidad Admin de Room. Solo una validación exitosa conmuta el estado a verdadero, otorgando acceso completo al menú de navegación inferior de la app.

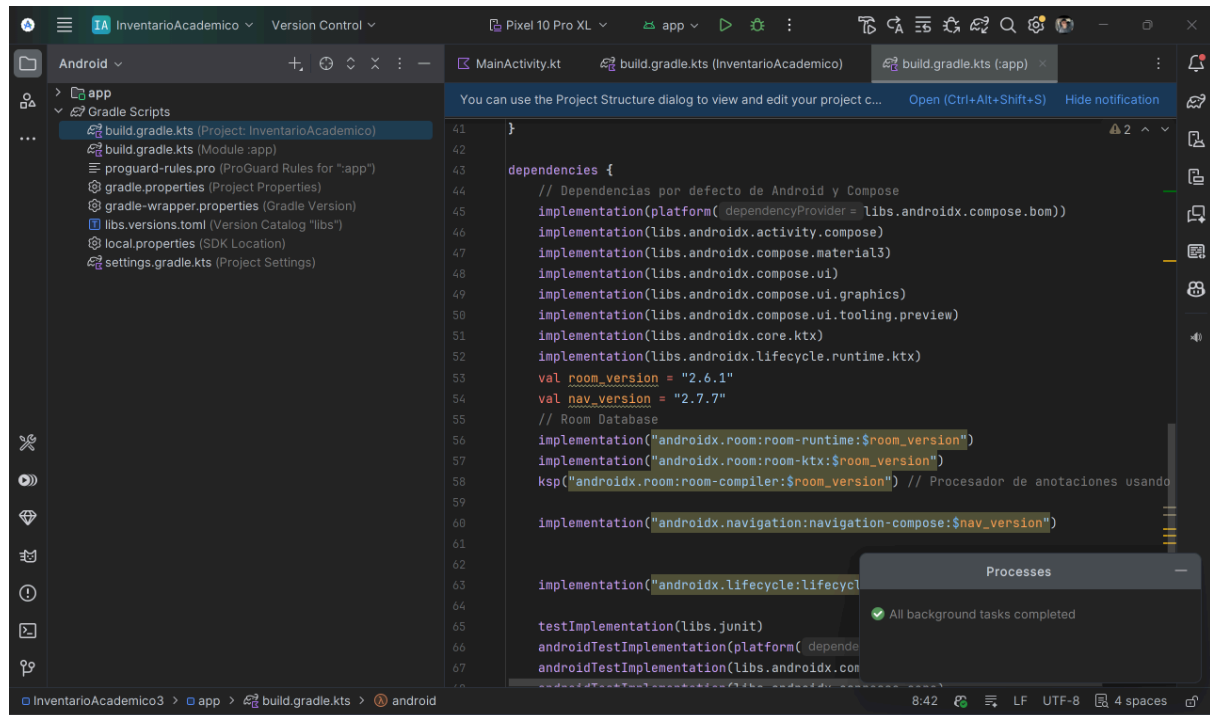
5. CONCLUSIONES TECNOLÓGICAS Y PERSPECTIVAS DE AMPLIACIÓN

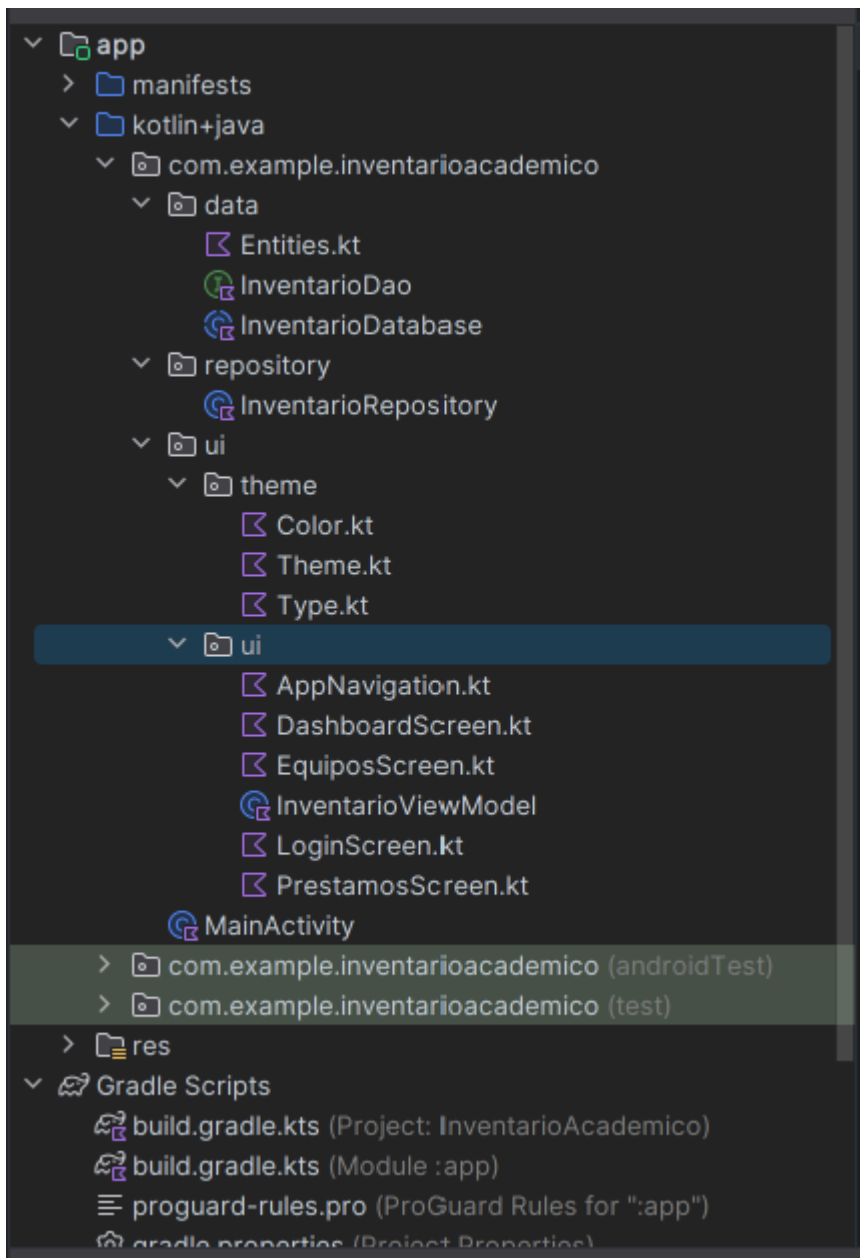
El desarrollo de *InventArk-UAM* demuestra la viabilidad y robustez de implementar soluciones empresariales medianas utilizando herramientas de desarrollo nativo moderno. La arquitectura modular adoptada mitiga errores comunes en aplicaciones Android como la recreación descontrolada de vistas o la pérdida de datos ante rotaciones de pantalla.

Ventajas Clave del Diseño:

1. Consumo Eficiente de Recursos: El uso de Room Database sobre una base SQLite local elimina la necesidad de conectividad constante a redes, permitiendo al personal de inventario operar en sótanos o laboratorios aislados.
2. Reactividad al 100%: Gracias a la adopción de **Flow** en Kotlin y los estados nativos de Compose, la sincronización entre la base de datos local y lo que ve el usuario en pantalla ocurre en milisegundos sin necesidad de refrescar la interfaz manualmente.
3. Portabilidad: El módulo de exportación CSV dota a la aplicación de una interoperabilidad inmediata con las herramientas de oficina tradicionales de la universidad.

1. Capturas de pantalla del sistema.





Base de datos Room funcional.

